The general registers of the EU (Execution Unit)

eAX – Accumulator register
- used by most instructions as one operands

eBX – The Base register

db, dw, dd, dq
1 1 1
16 32 64

a[7] = *(a+7)
=
T
base

eCX – Counter register
eDX – Data register
- used usually w/ EAX in computations where the result of an op. exceeds a dw

EDX : EAX

eSP – Stack Pointer
- contains a pointer to the element of the top of the stack

EBP - Base Pointer

- contains a pointer to the element of the bottom of the stack (the last one)

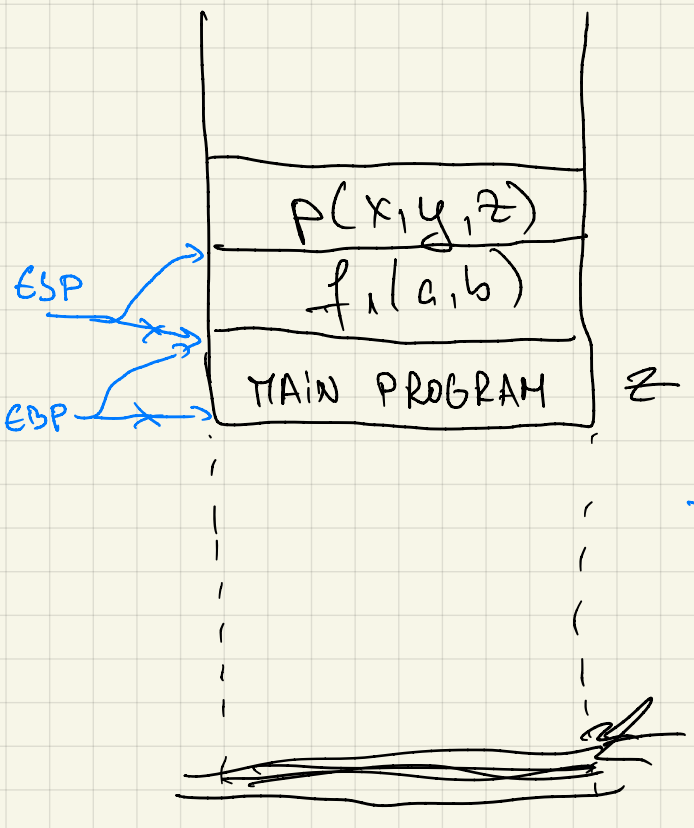
Stack => LIFO

SS - Stack Segment

of Microproc.

* Why 3 registers are related to the stack?

- we use stacks because of subroutines (functions etc.) and because that's how the code is written.



= activation record (info needed to run like input, return, etc.)
(stack frame)

The implementation of the running program.

SS

Runtime stack

- when f1(a,b) function is called, it becomes the top of the stack, execution of MP is suspended.

- the MP starts the execution & stops it always (=> stack is empty in the end)

* Why do we need 3 registers for the stack?
the role of ESP / EBP is to always define the currently executing stack frame

- when f1. is called => EBP is saved (in another stack)
  - then it is moved to the top.
    (when the new stack frame grows)
  - then ESP is moved to the top (same way)

- when we need to cleanup, we move everything where it was before mov EBP, ESP. etc

CS - code segment
DS - data segment
SS - stack segment
ES - extra segment

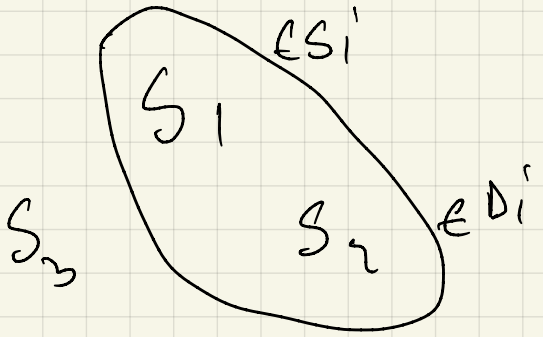
=> the 4 logic segments.

SS => pointer to the start of the Runtime Stack

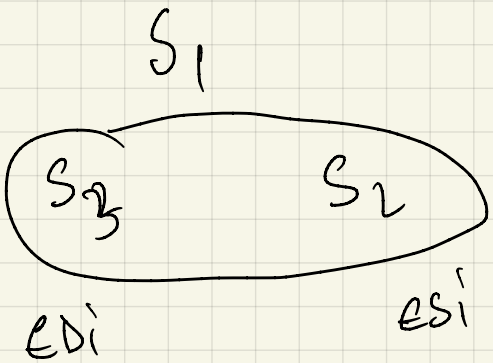
EDi – Destination Index (pointer) mov ESi, 7
ESi – Source Index / for strings / arrays
(pointer)

a[7] => the index "7" is controlled by those two registers

- Can I implement in asm algo that works with strings
- take in pairs of two.
- after your finish operation, you go to the next register



push ESi
push EDi
=> they go to memory

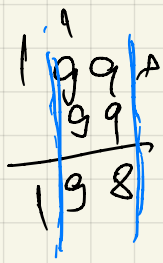


push ESi
push EDi

. . . .

- A flag is an indicator repr. on 1 bit.
- A configuration of the FLAGS register shows a synthetic overview of the execution of the each instruction.
- For x86 the EFLAGS register has 32 bits but, only only a one actually used.

CF - Carry Flag - transport flag
(PDI (last performed operation)
- will be set to 1 if I have a transport digit out of the representation domain



byte + byte = byte
dw + dw = dw...

Why they decided that in the case of + and - that the result to be mandatory to be represented on the same size?

Why do you think that ASM doesn't allow you to have instructions operands?
- 0 operands

- 1 operands
- 2 operands max.

the carry flag signals that the result overflowed & that we have an extra digit

- if WE HAVE 3 => C can be 0, 1, (2) PROBLEM, ↓ which is not good.

FLAG is 0/1.

A nr. in b_2 has a interpretation on b_{10} based on the sign.

Which is a very fast way to convert from b_2 to b_{10}

$b_2 \Rightarrow b_{16} \Rightarrow b_{10}$

CF FLAGS THE UNSIGNED OVERFLOW

```
mov ah, 93h
mov al, 73h
add ah, al      ; AH = 06h, CF = 1
mov al, ah
mov ah, 0
adc ah, 0
```

93h +
73h
106h